Market Risk Project

ESILV

Academic Year 2025-2026

Students:
Lilian Fages
Nel Choisnet Dupuij

Supervisor:
LEFEVRE Cassandre

Contents

1 Introduction **2**

2 Questions and Answers **2**

2.1 Question A (Ex2, part of Q1 and of Q2 of TD1) 2

2.1.1 a) Historical non-parametric VaR (biweight kernel KDE) 2

2.1.2 b) Out-of-sample exceedances (2017-2018) and validation 7

2.2 Question B (Ex2, Q5 of TD2) 13

2.3 Question C (Ex2, Q1 and Q2 of TD3) 15

2.3.1 a) Pickands tail-index estimation (upper and lower tails) 15

2.3.2 b) EVT-based VaR using the Pickands estimator (losses) 21

2.4 Question D — Estimation of Bouchaud's price impact model 25

2.4.1 Theoretical background 25

2.4.2 Estimation methodology 26

2.4.3 Results 29

2.4.4 Analysis and interpretation 30

2.4.5 Conclusion 31

2.5 Question E(a) — Multiresolution correlations (Haar wavelets) and the Epps effect 31

2.5.1 Average price and high-frequency returns 32

2.5.2 Haar multiresolution decomposition 32

2.5.3 Multiresolution covariance and correlation (course definition) 32

2.5.4 Results: multiresolution correlations and Epps effect 32

2.5.5 How to explain the Epps effect (microstructure mechanisms) 33

2.6 Question E(b) — Hurst exponents and annualized volatilities 34

3 Conclusion **37**

1 Introduction

This project will apply the main market risk tools that are introduced in the *Market Risk* course to real financial datasets (Natixis for example). We mainly focus on one-day risk measures and on the statistical and economic mechanisms that drive them, from the bulk of the distribution to the extreme tails, and from price dynamics to liquidity effects and multi-scale dependence.

We first estimate a *historical* one-day VaR for Natixis using a fully non-parametric approach based on the *biweight-kernel* KDE required in the statement, and we assess its out-of-sample performance over 2017-2018 via exception counts and standard coverage diagnostics. We then compute the corresponding Expected Shortfall to quantify tail severity beyond VaR. Next, we investigate extreme losses using EVT through the Pickands tail-index estimator and derive EVT-based VaR levels, showing the sensitivity to the threshold parameter. We also estimate Bouchaud's propagator model to quantify transient price impact and the concavity of volume dependence. Finally, using Haar wavelets on FX rates, we study multi-resolution correlations (and the Epps effect), and we estimate Hurst exponents to discuss volatility scaling and annualization beyond the Brownian benchmark.

2 Questions and Answers

2.1 Question A (Ex2, part of Q1 and of Q2 of TD1)

2.1.1 a) Historical non-parametric VaR (biweight kernel KDE)

This question asks for a *historical* one-day Value-at-Risk (VaR) computed from a *non-parametric* estimate of the return distribution, using the **biweight kernel** provided in the statement.

Data and returns (in-sample 2015-2016) We work with Natixis daily prices over the in-sample period January 2015 to December 2016 and compute daily log-returns $r_t = \log(P_t) - \log(P_{t-1})$. This yields $n = 512$ observations.

Biweight KDE and VaR definition : Let $X_1, \ldots, X_n$ denote the in-sample returns. The kernel density estimator is

$$\widehat{f}_h(x) = \frac{1}{nh} \sum_{i=1}^{n} K\left(\frac{x - X_i}{h}\right), \qquad K(u) = \frac{15}{16}\,(1-u^2)^2 \mathbf{1}_{\{|u|\le 1\}},$$

with bandwidth $h > 0$. The associated CDF estimator is

$$\widehat{F}_h(x) = \int_{-\infty}^{x} \widehat{f}_h(u)\,du = \frac{1}{n} \sum_{i=1}^{n} F_K\left(\frac{x - X_i}{h}\right),$$

where F_K is the CDF of the biweight kernel (implemented in closed form on $(-1, 1)$). The one-day VaR at confidence level α is the left-tail quantile (reported as a *positive loss*):

$$\mathrm{VaR}_\alpha = -\,\widehat{F}_h^{-1}(1-\alpha).$$

In practice, we compute $\widehat{F}_h^{-1}(1-\alpha)$ by bisection.

Bandwidth choice and sensitivity The bandwidth h controls the smoothing of the KDE: small h yields a rough estimate, while large h may oversmooth the distribution and affect tail risk estimates. Following the course recommendations, we use a robust Silverman/Scott rule-of-thumb,

$$h_0 = 0.9 \min\left(\widehat{\sigma}, \frac{\mathrm{IQR}}{1.34}\right) n^{-1/5}.$$

To assess robustness, we plot the KDE for $h = m\,h_0$ with $m \in \{0.5, 0.75, 1, 1.5, 2\}$.

Implementation Listing 1 contains the Python code used to construct daily returns, estimate the biweight-kernel KDE, invert the estimated CDF by bisection to obtain $\widehat{F}_h^{-1}(1-\alpha)$, and produce the accompanying figures.

Listing 1: Historical VaR via biweight-kernel KDE (self-contained cell, in-sample 2015-2016).

```python
# ==============================================================
# Q1(A-a)  Historical 1-day VaR using a NON-PARAMETRIC KDE
# with the BIWEIGHT kernel: K(u)=15/16(1-u^2)^2 1{|u|<=1}
# ==============================================================

import os, math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# -------------
# Parameters (edit easily)
# -------------
ALPHA = 0.99
RETURN_TYPE = "log"
PATH = "Natixis_Stock.csv"
DATE_START = "2015-01-01"
DATE_END = "2016-12-31"
H_MULTS = [0.5, 0.75, 1.0, 1.5, 2.0]
FIG_DIR = "figures"
os.makedirs(FIG_DIR, exist_ok=True)

def read_price_file(path: str) -> pd.DataFrame:
    df = pd.read_csv(path)
    df = df.rename(columns={df.columns[0]: "Date", df.columns[1]: "Price"})
    df["Date"] = pd.to_datetime(df["Date"], dayfirst=True, errors="coerce")
    df["Price"] = pd.to_numeric(df["Price"], errors="coerce")
    df = df.dropna(subset=["Date", "Price"]).sort_values("Date").reset_index(drop=True)
    return df

def compute_returns(df: pd.DataFrame, return_type: str = "log") -> pd.Series:
    p = df["Price"].astype(float).values
    if return_type == "log":
```

```python
        if np.any(p <= 0):
            raise ValueError("Log-returns require strictly positive prices.")
        r = np.diff(np.log(p))
    elif return_type == "simple":
        r = np.diff(p) / p[:-1]
    else:
        raise ValueError("RETURN_TYPE must be 'log' or 'simple'.")
    return pd.Series(r, index=df["Date"].iloc[1:], name="Return")

def biweight_kernel(u: np.ndarray) -> np.ndarray:
    u = np.asarray(u, dtype=float)
    out = np.zeros_like(u)
    m = np.abs(u) <= 1.0
    um = u[m]
    out[m] = (15.0/16.0) * (1.0 - um**2)**2
    return out

def biweight_kernel_cdf(u: np.ndarray) -> np.ndarray:
    u = np.asarray(u, dtype=float)
    out = np.zeros_like(u)
    out[u >= 1.0] = 1.0
    mid = (u > -1.0) & (u < 1.0)
    um = u[mid]
    poly = um - (2.0/3.0)*um**3 + (1.0/5.0)*um**5
    out[mid] = (15.0/16.0) * (poly + 8.0/15.0)
    return out

def bandwidth_rule_of_thumb(x: np.ndarray) -> float:
    x = np.asarray(x, dtype=float)
    n = x.size
    std = np.std(x, ddof=1)
    q75, q25 = np.percentile(x, [75, 25])
    iqr = q75 - q25
    scale = min(std, iqr/1.34) if iqr > 0 else std
    if scale <= 0:
        return 0.0
    return 0.9 * scale * (n ** (-1.0/5.0))

def kde_biweight_pdf(x_grid: np.ndarray, data: np.ndarray, h: float) -> np.ndarray:
    u = (x_grid[:, None] - data[None, :]) / h
    return np.mean(biweight_kernel(u), axis=1) / h

def kde_biweight_cdf(t: float, data: np.ndarray, h: float) -> float:
    u = (t - data) / h
    return float(np.mean(biweight_kernel_cdf(u)))

def kde_biweight_quantile_left(data: np.ndarray, h: float, p: float, n_iter: int =
    120) -> float:
```

```python
    lo = float(np.min(data) - h)
    hi = float(np.max(data) + h)
    for _ in range(n_iter):
        mid = 0.5*(lo + hi)
        if kde_biweight_cdf(mid, data, h) < p:
            lo = mid
        else:
            hi = mid
    return 0.5*(lo + hi)

# -- Load, returns, in-sample --
prices = read_price_file(PATH)
rets_all = compute_returns(prices, RETURN_TYPE)
rets = rets_all.loc[DATE_START:DATE_END].dropna()

x = rets.values.astype(float)
n = len(x)
h0 = bandwidth_rule_of_thumb(x)

p_left = 1.0 - ALPHA
q_left = kde_biweight_quantile_left(x, h0, p_left)
VaR_kde = float(-q_left)

out = pd.DataFrame([{
    "Alpha": ALPHA,
    "Obs (2015-2016)": n,
    "Bandwidth h0": h0,
    "KDE biweight VaR (1-day, loss>0)": VaR_kde
}])
print(out)

# -- Figure 1: bandwidth sensitivity --
std = np.std(x, ddof=1)
x_grid = np.linspace(np.min(x) - 4*std, np.max(x) + 4*std, 800)

plt.figure(figsize=(9, 5))
for m in H_MULTS:
    h = m * h0
    plt.plot(x_grid, kde_biweight_pdf(x_grid, x, h), label=f"h={m:.2f}h0")
plt.title("Biweight KDE of Natixis daily returns (20152016): bandwidth sensitivity")
plt.xlabel("Return"); plt.ylabel("Density")
plt.grid(True); plt.legend()
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "A_a_kde_biweight_bandwidth.pdf"),
    bbox_inches="tight")
plt.show()

# -- Figure 2: histogram + KDE + VaR --
```

```python
plt.figure(figsize=(9, 5))
plt.hist(x, bins=60, density=True, alpha=0.35, edgecolor="none")
plt.plot(x_grid, kde_biweight_pdf(x_grid, x, h0), linewidth=2, label="Biweight KDE (h=h0)")
plt.axvline(q_left, linestyle="-", linewidth=2, label=f"q_(1-)={q_left:.5f}")
plt.title(f"Natixis daily returns (20152016): Biweight KDE and VaR (={ALPHA:.2f})")
plt.xlabel("Return"); plt.ylabel("Density")
plt.grid(True); plt.legend()
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "A_a_var_kde_biweight_overlay.pdf"), bbox_inches="tight")
plt.show()

print(f"VaR_KDE (biweight) at alpha={ALPHA:.2f}: {VaR_kde:.6f} (positive loss)")
```

Results For $\alpha = 0.99$, the estimation yields $h_0 \approx 0.005441$ and

$$\widehat{q}_{1-\alpha} \approx -0.05923, \qquad \widehat{\mathrm{VaR}}_{0.99} \approx 0.05923.$$

Hence, the 99% one-day VaR corresponds to an estimated loss of about **5.9%** of the stock value over one day (in return space). Expressed in monetary units, this threshold would be obtained by multiplying by the current price level.

α	n (2015-2016)	h_0	$\widehat{q}_{1-\alpha}$	$\widehat{\mathrm{VaR}}_\alpha$
0.99	512	0.005441	-0.05923	0.05923

Table 1: Biweight-KDE historical one-day VaR on Natixis log-returns (in-sample 2015-2016). VaR is reported as a positive loss.

Illustrations. Figure 1 illustrates the sensitivity of the biweight KDE to the bandwidth choice around h_0. Figure 2 overlays the fitted density (at $h = h_0$) with the left-tail quantile defining VaR.

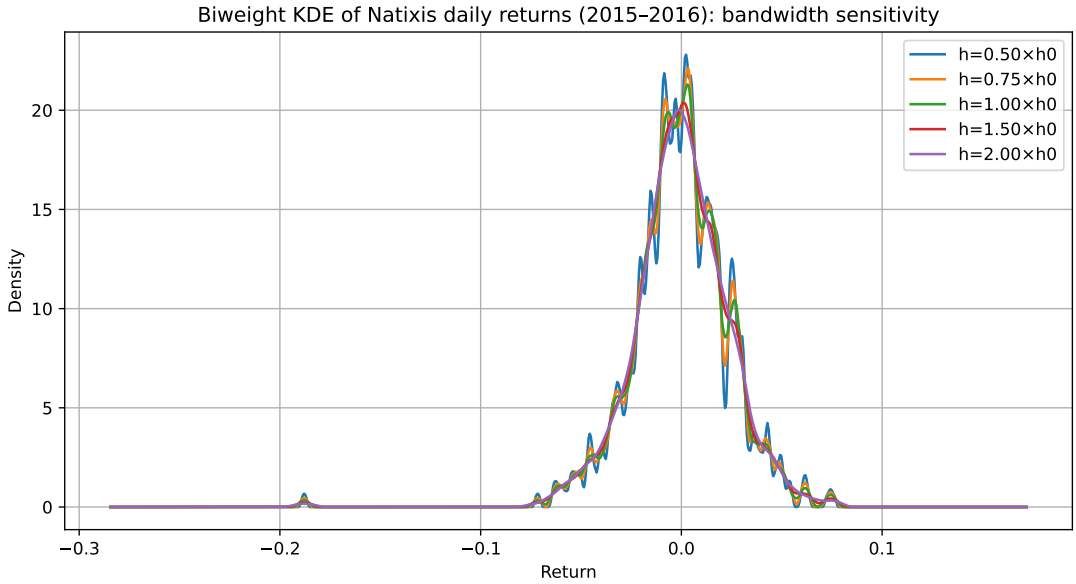

Figure 1: Biweight KDE of Natixis daily log-returns (2015-2016) for several bandwidths $h = m\,h_0$. Smaller h produces a more irregular estimate, whereas larger h yields a smoother density.

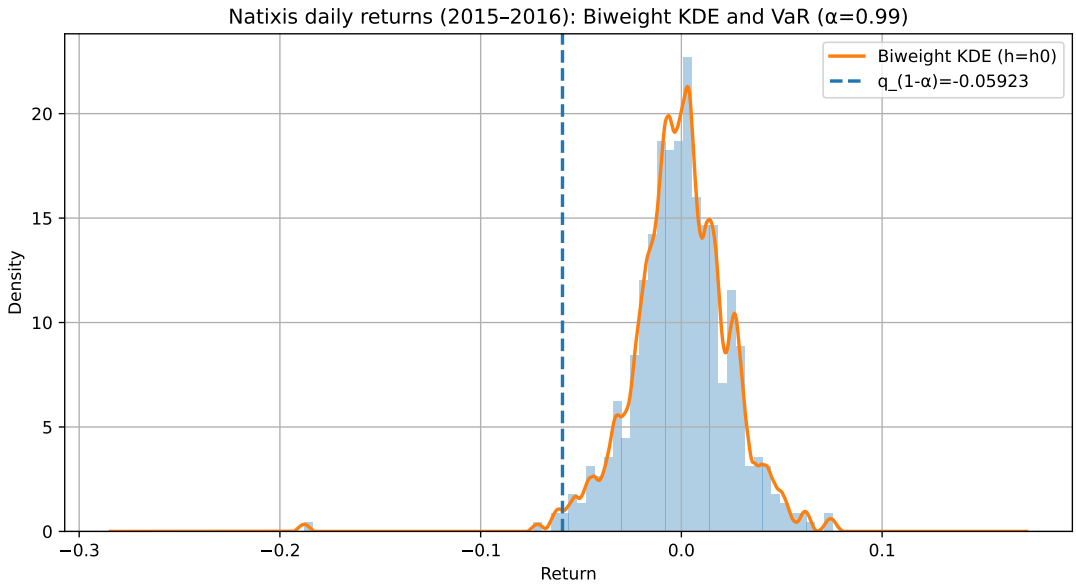

Figure 2: Natixis daily log-returns (2015-2016): histogram, biweight KDE (with $h = h_0$), and left-tail quantile $\widehat{q}_{1-\alpha}$ (dashed line) defining $\widehat{\mathrm{VaR}}_\alpha = -\widehat{q}_{1-\alpha}$ for $\alpha = 0.99$.

This in-sample VaR threshold is the reference level used in part (b) to evaluate the out-of-sample exceedance frequency over 2017-2018.

2.1.2 b) Out-of-sample exceedances (2017-2018) and validation

We now assess the out-of-sample performance of the *biweight-KDE historical VaR* calibrated on the in-sample period 2015-2016 (Question A-a). Let r_t denote the one-day return and let $q_{1-\alpha}$ be the estimated left-tail quantile returned by the KDE procedure. A VaR exception (also called

a *hit*) occurs when

$$r_t < q_{1-\alpha} \qquad \Longleftrightarrow \qquad \ell_t = -r_t > \mathrm{VaR}_\alpha,$$

where $\mathrm{VaR}_\alpha = -q_{1-\alpha} > 0$ is expressed as a positive loss.

Implementation Listing 2 contains the Python cell used to (i) extract 2017-2018 returns, (ii) compute the number of exceptions k and the hit rate $\widehat{p} = k/N$ relative to the fixed threshold $q_{1-\alpha}$ estimated in 2015-2016, (iii) run the unconditional coverage (Kupiec) test, and (iv) generate the two diagnostic figures (time series of returns with exceptions, and cumulative exceptions with a binomial envelope).

Listing 2: Backtesting of the BIWEIGHT-KDE VaR over 2017-2018 (depends only on outputs from A-a).

```python
# ==============================================================
# Q(A-b)  Backtesting (20172018) for the BIWEIGHT-KDE VaR
#
# Depends ONLY on Q(A-a) outputs:
# - rets_all : all daily returns (pd.Series), indexed by Date
# - ALPHA : confidence level (e.g., 0.99)
# - q_left OR var_kde_loss:
# q_left = estimated return quantile q_{1-} (negative)
# var_kde_loss= VaR as positive loss (=-q_left)
# ==============================================================

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# ------------
# 0) Sanity checks
# ------------
required = ["rets_all", "ALPHA"]
missing = [v for v in required if v not in globals()]
if missing:
    raise NameError(f"Missing variables from Q(A-a): {missing}. Run Q(A-a) first.")

if "q_left" in globals():
    q_var = float(q_left) # return threshold: q_{1-} < 0
    var_loss = float(-q_var) # VaR as positive loss
elif "var_kde_loss" in globals():
    var_loss = float(var_kde_loss) # VaR as positive loss
    q_var = float(-var_loss) # return threshold
else:
    raise NameError("Need either 'q_left' or 'var_kde_loss' from Q(A-a).")

# Output folder for figures (Windows-safe string)
FIG_DIR = r"C:\Users\lil16\Desktop\figures"
```

```python
os.makedirs(FIG_DIR, exist_ok=True)

# ------------
# 1) Out-of-sample window
# ------------
test = rets_all.loc["2017-01-01":"2018-12-31"].dropna()
y = test.values.astype(float)
N = len(y)
if N < 30:
    raise ValueError("Out-of-sample window too small; check dates and data.")

# Hit/exception if return < q_{1-} <=> loss > VaR
hits = (y < q_var)
k = int(hits.sum())
hit_rate = k / N
p0 = 1.0 - float(ALPHA) # nominal exception probability

# ------------
# 2) Kupiec UC test + exact CI on hit rate
# ------------
eps = 1e-12
phat = max(min(hit_rate, 1 - eps), eps)
p0c = max(min(p0, 1 - eps), eps)

LR_uc = -2.0 * (
    (N - k) * np.log((1 - p0c) / (1 - phat)) + k * np.log(p0c / phat)
)

pval_uc = None
ci_rate = None

try:
    from scipy.stats import chi2, beta, binom
    pval_uc = 1.0 - chi2.cdf(LR_uc, df=1)

    # ClopperPearson exact 95% CI for Bernoulli rate
    alpha_ci = 0.05
    if k == 0:
        ci_lo = 0.0
        ci_hi = beta.ppf(1 - alpha_ci/2, k + 1, N - k)
    elif k == N:
        ci_lo = beta.ppf(alpha_ci/2, k, N - k + 1)
        ci_hi = 1.0
    else:
        ci_lo = beta.ppf(alpha_ci/2, k, N - k + 1)
        ci_hi = beta.ppf(1 - alpha_ci/2, k + 1, N - k)
    ci_rate = (float(ci_lo), float(ci_hi))

```

```python
except Exception:
    # Fallback: normal approx CI if scipy is not available
    z = 1.96
    se = np.sqrt(max(hit_rate * (1 - hit_rate), eps) / N)
    ci_lo = max(0.0, hit_rate - z * se)
    ci_hi = min(1.0, hit_rate + z * se)
    ci_rate = (ci_lo, ci_hi)
    binom = None

# Summary table
summary = pd.DataFrame([{
    "alpha": float(ALPHA),
    "VaR (loss>0)": var_loss,
    "q_(1-alpha) (return)": q_var,
    "N (2017-2018)": N,
    "k exceptions": k,
    "hit rate k/N": hit_rate,
    "expected count N*(1-alpha)": N * p0,
    "nominal 1-alpha": p0,
    "95% CI hit rate": f"[{ci_rate[0]:.4f}, {ci_rate[1]:.4f}]",
    "Kupiec LR_uc": LR_uc,
    "Kupiec p-value": pval_uc
}])

display(summary)

# ------------
# 3) Figure 1  Returns + VaR threshold + exceptions (KEEP)
# ------------
plt.figure(figsize=(11, 4.8))
plt.plot(test.index, y, linewidth=0.8, label="Daily returns (20172018)")
plt.axhline(q_var, linestyle="-", linewidth=1.5,
           label=f"VaR threshold $q_{{1-\\alpha}}$ = {q_var:.5f}")
plt.scatter(test.index[hits], y[hits], s=22, label=f"Exceptions (k={k})")
plt.title(f"Natixis returns (20172018) and BIWEIGHT-KDE VaR threshold
    ($\\alpha$={ALPHA})")
plt.xlabel("Date")
plt.ylabel("Return")
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
f1_pdf = os.path.join(FIG_DIR, "A_b_exceedances_timeseries.pdf")
plt.savefig(f1_pdf, bbox_inches="tight")
plt.show()

# ------------
# 4) Figure 2  Cumulative exceptions with 95% Binomial envelope (COURSE-STYLE)
# ------------
```

```python
cum_hits = np.cumsum(hits.astype(int))
t = np.arange(1, N + 1)
expected_line = p0 * t

# Pointwise 95% envelope for K_t ~ Binomial(t, p0)
if binom is not None:
    lo = binom.ppf(0.025, t, p0)
    hi = binom.ppf(0.975, t, p0)
else:
    # Normal approximation fallback
    z = 1.96
    sd = np.sqrt(t * p0 * (1 - p0))
    lo = np.maximum(0.0, expected_line - z * sd)
    hi = expected_line + z * sd

plt.figure(figsize=(11, 4.4))
plt.plot(test.index, cum_hits, linewidth=2.0, label="Cumulative exceptions")
plt.plot(test.index, expected_line, linestyle="-", linewidth=1.8, label="Expected cumulative count")
plt.fill_between(test.index, lo, hi, alpha=0.2, label="95% Binomial envelope (coverage)")
plt.title(f"Cumulative exceptions with Binomial envelope ($\\alpha$={ALPHA}, nominal rate={p0:.2%})")
plt.xlabel("Date")
plt.ylabel("Cumulative count")
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
f2_pdf = os.path.join(FIG_DIR, "A_b_exceedances_cumulative_envelope.pdf")
plt.savefig(f2_pdf, bbox_inches="tight")
plt.show()

print(f"Saved figures:\n- {f1_pdf}\n- {f2_pdf}")
print(f"Backtest: k={k}, N={N}, hit rate={hit_rate:.2%} vs nominal {p0:.2%}.")
if pval_uc is not None:
    print(f"Kupiec UC test: LR={LR_uc:.3f}, p-value={pval_uc:.3f}")
```

Backtest results (2017-2018) At confidence level $\alpha = 0.99$, the nominal exception probability is $p_0 = 1 - \alpha = 1\%$. Over the out-of-sample window 2017-2018 we have $N = 510$ daily returns, hence the expected number of exceptions under correct calibration is $Np_0 \simeq 5.1$.

With the biweight-KDE threshold estimated in-sample, we obtain:

$$\mathrm{VaR}_{0.99} \approx 0.059231, \qquad q_{1-\alpha} \approx -0.059231, \qquad k = 1, \qquad \widehat{p} = \frac{k}{N} \approx 0.196\%.$$

Therefore, the observed hit rate is well below the nominal 1%, indicating that the in-sample VaR is *conservative* over 2017-2018.

Statistical validation (Kupiec unconditional coverage) Following the courses backtesting framework, we give the Kupiec unconditional coverage statistic: (LR_{uc}, asymptotically $\chi^2(1)$ under correct coverage). For our sample we have $\mathrm{LR}_{uc} \approx 4.975$ with p-value ≈ 0.026, which leads to a rejection of the null hypothesis of correct unconditional coverage at the 5% level. Here the rejection is driven by *too few exceptions* (relative to $p_0 = 1\%$), consistent with a VaR that overestimates tail risk in 2017-2018.

This outcome should be looked at in light of (i) the moderate sample size (two years), and (ii) the fact that a fixed VaR calibrated on 2015-2016 may face a volatility/regime shift in 2017-2018. From a risk-management perspective the model remains *safe* (conservative), but it is not perfectly calibrated over the test window according to the unconditional coverage criterion.

Diagnostic figures Figure 3 displays 2017-2018 returns together with the VaR threshold and the identified exceptions. Figure 4 compares cumulative exceptions to the nominal expectation and adds a 95% binomial coverage envelope, showing that the observed exception path remains below the expected line and is compatible with a conservative VaR.

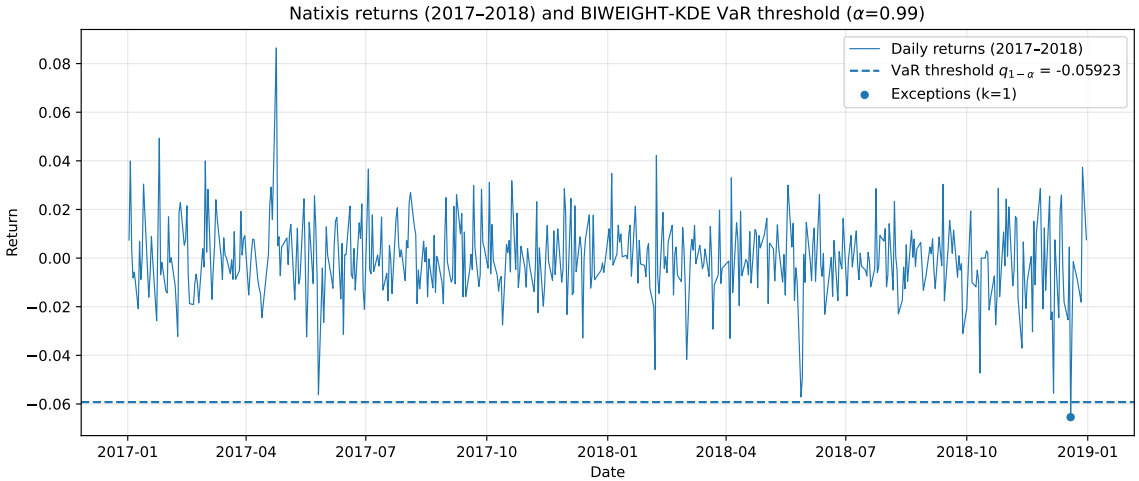

Figure 3: Out-of-sample returns (2017-2018) with the fixed BIWEIGHT-KDE VaR threshold (estimated on 2015-2016) and highlighted exceptions ($k = 1$).

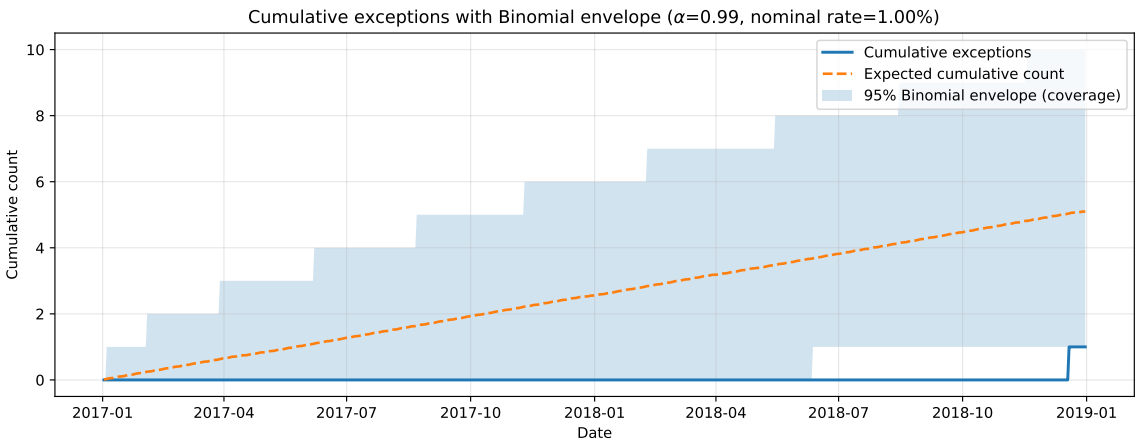

Figure 4: Cumulative exceptions versus nominal expectation ($\alpha = 0.99$) with a 95% binomial envelope under unconditional coverage.

Conclusion (answer to Question A-b) Over January 2017-December 2018, the proportion of daily returns that exceed the VaR threshold estimated on 2015-2016 is

$$\widehat{p} = \frac{k}{N} = \frac{1}{510} \approx 0.20\%,$$

to be compared with the nominal exceedance probability $1 - \alpha = 1\%$ (so the nominal expected count is $Np_0 \simeq 5.1$). Hence, the biweight-KDE historical VaR calibrated on 2015-2016 produces *fewer* exceptions than expected in 2017-2018: it is conservative on this out-of-sample period. In terms of validation, the unconditional coverage (Kupiec) test yields a p-value ≈ 0.026, which rejects correct coverage at the 5% level; the rejection is driven by *under*-exceedance rather than excessive risk. We therefore conclude that the non-parametric KDE VaR provides a stable and prudent one-day risk measure, but that its calibration is not fully accurate over 2017-2018 (it tends to overestimate tail risk in this window).

2.2 Question B (Ex2, Q5 of TD2)

Question B asks for the **Expected Shortfall (ES)** associated with the one-day **VaR** computed in Question A, and for a comparison between these two risk measures. We therefore keep the same confidence level α and the same in-sample window (2015-2016), and we use the **KDE-biweight VaR cutoff** from Question A to define the tail events over which ES is averaged.

Definition (returns and positive losses) Let r denote the daily return and define the positive loss as $L = -r$. For a confidence level $\alpha \in (0, 1)$, letting $p = 1 - \alpha$ and $q_p = F_r^{-1}(p)$, we recall

$$\mathrm{VaR}_\alpha = -q_p, \qquad \mathrm{ES}_\alpha = \mathbb{E}[L \,|\, L \geq \mathrm{VaR}_\alpha] = -\mathbb{E}[r \,|\, r \leq q_p]\,.$$

VaR provides a quantile-based loss threshold, while ES measures the *average loss severity* beyond that threshold.

Estimation principle In Question A, the left-tail cutoff $q_{1-\alpha}$ is obtained from the biweight-KDE estimate of the return distribution. Given this cutoff, we estimate ES as the *sample mean* of returns below $q_{1-\alpha}$ (equivalently, the mean of losses above VaR_α). We keep the KDE-biweight VaR cutoff from Question A to define tail events, and estimate ES as the historical (sample) conditional mean of losses beyond this cutoff.

Listing 3: Question B — Expected Shortfall for the VaR from Question A (KDE biweight)

```python
import numpy as np
import pandas as pd

# Assumes already available from Question A-a (same in-sample, same KDE cutoff):
# - rets : pd.Series of returns over 2015-2016 (in-sample)
# - ALPHA : confidence level (e.g., 0.99)
# - q_left : KDE-biweight left-tail quantile of returns at p=1-ALPHA (so q_left<0)

# Tail returns beyond the KDE VaR threshold (historical ES conditional on KDE cutoff)
tail = rets.dropna().astype(float).values
tail = tail[tail <= q_left]

ES_return = float(tail.mean()) # negative
ES_loss = float(-ES_return) # positive loss

VaR_loss = float(-q_left)


answer_B = pd.DataFrame([{
    "Alpha": ALPHA,
    "VaR_QA (KDE biweight, loss)": VaR_loss,
    "ES_QB (KDE biweight, loss)": ES_loss,
    "ES - VaR": ES_loss - VaR_loss,
    "ES / VaR": ES_loss / VaR_loss
}])

answer_B
```

Numerical results and comparison with VaR Table 2 reports the one-day VaR from Question A and the corresponding ES at the same confidence level. All quantities are expressed as **positive losses**.

Table 2: Expected Shortfall corresponding to the KDE-biweight VaR (Question A), one-day horizon.

α	$\mathrm{VaR}_\alpha^{\mathrm{KDE}}$	ES_α	$\mathrm{ES} - \mathrm{VaR}$	ES/VaR
0.99	0.0592	0.0889	0.0297	1.50

As expected, ES_α is larger than VaR_α because it averages losses in the tail rather than

reporting only a cutoff. Here, ES/VaR ≈ 1.50 indicates that, conditional on being in the worst 1% of daily returns, the average loss is about **50% larger** than the VaR threshold.

Conclusion The ES computed at the same confidence level as Question A is **strictly more conservative** than VaR and provides additional information about the *severity* of extreme losses. In this dataset, the sizable ES-VaR gap shows that VaR alone understates the average magnitude of tail losses, which motivates reporting ES as a complementary risk measure.

2.3 Question C (Ex2, Q1 and Q2 of TD3)

2.3.1 a) Pickands tail-index estimation (upper and lower tails)

This question aims at characterizing the tail behaviour of Natixis daily returns through an estimate of the *extreme value index* (tail index) ξ. Throughout Question C, daily returns are assumed to be independent and identically distributed. This assumption underlies both the Pickands tail-index estimation and the EVT-based VaR extrapolation, and is standard in first-order EVT applications, despite being a simplification in the presence of volatility clustering.

Following the course, we use the Pickands estimator based on order statistics. For an i.i.d. sample $(X_i)_{i=1}^n$ with order statistics $X_{1:n} \leq \cdots \leq X_{n:n}$, the Pickands estimator for the **right tail** is, for any integer k such that $4k \leq n$,

$$\widehat{\xi}_{\mathrm{P}}(k) = \frac{1}{\log 2} \log\left(\frac{X_{n-k+1:n} - X_{n-2k+1:n}}{X_{n-2k+1:n} - X_{n-4k+1:n}}\right). \tag{1}$$

To study **both sides** of the return distribution R_t, we apply (1) separately to: (i) the **upper tail** using the gains subsample $\{R_t : R_t > 0\}$, and (ii) the **lower tail** using the **loss magnitudes** $\{-R_t : R_t < 0\}$ so that we again analyse a right tail. As discussed in the course, the choice of the threshold parameter k is critical: small k typically yields high variance, while large k may introduce bias. We therefore report the standard reference rules $k \in \{\lfloor \log n \rfloor, \lfloor \sqrt{n} \rfloor, \lfloor 0.05n \rfloor\}$ and complement them with stability plots $\widehat{\xi}_{\mathrm{P}}(k)$ versus k.

Implementation Listing 4 provides the Python implementation used to build the subsamples (gains and loss magnitudes), compute $\widehat{\xi}_{\mathrm{P}}(k)$ on an admissible grid (constraint $4k \leq n$), and generate the figures and summary table. As an additional simple diagnostic, we overlay a rolling-median smoothing to facilitate the visual identification of stable regions and we propose a heuristic $k^\star$ located in a locally stable area. This $k^\star$ is only used as a cross-check and does not replace the standard choices.

Listing 4: Pickands tail-index estimation (upper tail on gains and lower tail on loss magnitudes) with stability plots and a heuristic stable choice $k^\star$.

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from pathlib import Path
try:
```

```python
    from IPython.display import display
except Exception:
    display = print

# ------------
# 0) Inputs
# ------------
START_DATE = "2015-01-02"
END_DATE = "2018-12-31"

FIG_DIR = Path(r"C:\Users\lil16\Desktop\figures")
FIG_DIR.mkdir(parents=True, exist_ok=True)

# ------------
# 1) Load sample
# ------------
assert "rets_all" in globals(), "You must have `rets_all` (daily returns) defined before running this cell."

r = rets_all.loc[START_DATE:END_DATE].dropna()
if isinstance(r, pd.DataFrame):
    if r.shape[1] != 1:
        raise ValueError("`rets_all` is a DataFrame with multiple columns; provide a single return series.")
    r = r.iloc[:, 0]
r = r.astype(float)

# EVT tails (right-tail variables)
gains = r[r > 0].values
losses = (-r[r < 0]).values

if len(gains) < 50 or len(losses) < 50:
    raise ValueError(f"Not enough observations in one tail: #gains={len(gains)}, #losses={len(losses)}.")

print(f"Sample window: {START_DATE}  {END_DATE}")
print(f"Total obs: {len(r)} | Gains: {len(gains)} | Losses: {len(losses)}")

# ------------
# 2) Pickands estimator (right tail)
# ------------
def pickands_xi_right_tail(sample: np.ndarray, k: int) -> float:
    """
    Pickands estimator of the extreme value index xi for the RIGHT tail.
    Requires 4k < n (strict) to have X_{n-4k} well-defined.
    """
    x = np.asarray(sample, dtype=float)
    n = x.size
```

```python
    if k < 1 or 4 * k >= n:
        return np.nan

    xs = np.sort(x)
    x1 = xs[n - k]
    x2 = xs[n - 2 * k]
    x4 = xs[n - 4 * k]

    num = x1 - x2
    den = x2 - x4
    if num <= 0 or den <= 0:
        return np.nan

    return float(np.log(num / den) / np.log(2.0))

def make_k_grid(n: int, frac_max: float = 0.10, k_min: int = 5) -> np.ndarray:
    """
    Stability grid of k satisfying:
      - k >= k_min
      - k <= frac_max * n
      - 4k < n
    """
    k_max = min(int(frac_max * n), (n // 4) - 1)
    k_max = max(k_max, k_min + 5)
    k = np.arange(k_min, k_max + 1)
    k = k[4 * k < n]
    return k

def adjust_k(k: int, n: int, k_floor: int = 5) -> int:
    """Ensure k is admissible: k >= k_floor and 4k < n."""
    k = int(k)
    if k < k_floor:
        k = k_floor
    while 4 * k >= n and k > k_floor:
        k -= 1
    return k

def choose_k_by_stability(k_grid: np.ndarray, xi: np.ndarray, window: int = 9):
    """
    Heuristic choice: minimize rolling std on the stability plot.
    Returns (k_star, xi_star).
    """
    s = pd.Series(xi, index=k_grid).dropna()
    if len(s) < window:
        return None, None
    roll_std = s.rolling(window=window, center=True).std()
    k_star = int(roll_std.idxmin())
    return k_star, float(s.loc[k_star])
```

```python

# ------------
# 3) Compute stability curves + summary
# ------------
rows = []
fig_paths = []

for tail_name, sample, fname in [
    ("Upper tail (gains)", gains, "C_a_pickands_gains.pdf"),
    ("Lower tail (loss magnitudes)", losses, "C_a_pickands_losses.pdf")
]:
    n = len(sample)
    k_grid = make_k_grid(n, frac_max=0.10, k_min=5)
    xi = np.array([pickands_xi_right_tail(sample, k) for k in k_grid], dtype=float)

    k_log = adjust_k(int(np.log(n)), n)
    k_sqrt = adjust_k(int(np.sqrt(n)), n)
    k_5pc = adjust_k(int(0.05 * n), n)

    k_star, xi_star = choose_k_by_stability(k_grid, xi, window=9)

    rows.append({
        "Tail": tail_name,
        "n": n,
        "k_log(n)": k_log, "xi(k_log)": pickands_xi_right_tail(sample, k_log),
        "k_sqrt(n)": k_sqrt, "xi(k_sqrt)": pickands_xi_right_tail(sample, k_sqrt),
        "k_5% n": k_5pc, "xi(k_5%)": pickands_xi_right_tail(sample, k_5pc),
        "k_star": k_star, "xi(k_star)": xi_star
    })

    # rolling median (visual aid)
    s = pd.Series(xi, index=k_grid)
    smooth = s.rolling(window=9, center=True, min_periods=5).median()

    plt.figure(figsize=(8.5, 4.8))
    plt.plot(k_grid, xi, linewidth=1.2,
        label=r"$\widehat{\xi}_{\mathrm{Pickands}}(k)$")
    plt.plot(smooth.index, smooth.values, linewidth=2.0, label="Rolling median
        (window=9)")
    plt.axhline(0.0, linestyle="-", linewidth=1.0)

    plt.axvline(k_log, linestyle=":", linewidth=1.2, label=r"$k=\lfloor \log
        n\rfloor$")
    plt.axvline(k_sqrt, linestyle="-", linewidth=1.2, label=r"$k=\lfloor
        \sqrt{n}\rfloor$")
    plt.axvline(k_5pc, linestyle="-.", linewidth=1.2, label=r"$k=\lfloor
        0.05n\rfloor$")
    if k_star is not None:
```

```
        plt.axvline(k_star, linestyle="-", linewidth=1.0, alpha=0.9, label=r"$k^\star$ (stable)")

    plt.xlabel("k (number of top order statistics)")
    plt.ylabel(r"Pickands tail index estimate $\widehat{\xi}$")
    plt.title(f"Pickands stability plot  {tail_name}")
    plt.grid(True, alpha=0.3)
    plt.legend()
    plt.tight_layout()

    fpath = FIG_DIR / fname
    plt.savefig(fpath, bbox_inches="tight")
    plt.show()

    fig_paths.append(str(fpath))

summary_df = pd.DataFrame(rows)
display(summary_df)

print("\nSaved figures:")
for p in fig_paths:
    print(" -", p)
```

Stability plots Over the 2015-2018 window, the sample contains 1022 daily returns, split into 498 gains and 519 losses (in magnitude). Figures 5-6 display $\widehat{\xi}_{\mathrm{P}}(k)$ across admissible thresholds k (with $4k \leq n$), together with the rolling-median smoothing and vertical markers for the reference values and the proposed $k^\star$.

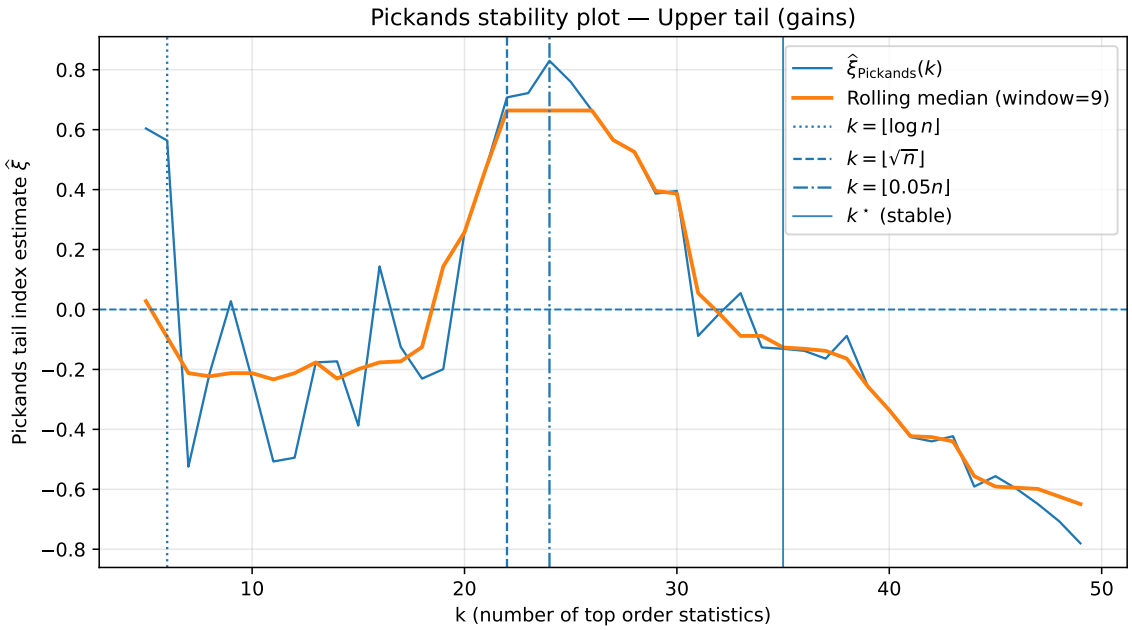

Figure 5: Pickands stability plot for the **upper tail** (gains). The rolling median is shown as a visual aid; vertical markers indicate $k = \lfloor \log n \rfloor$, $k = \lfloor \sqrt{n} \rfloor$, $k = \lfloor 0.05n \rfloor$ and the heuristic $k^\star$.

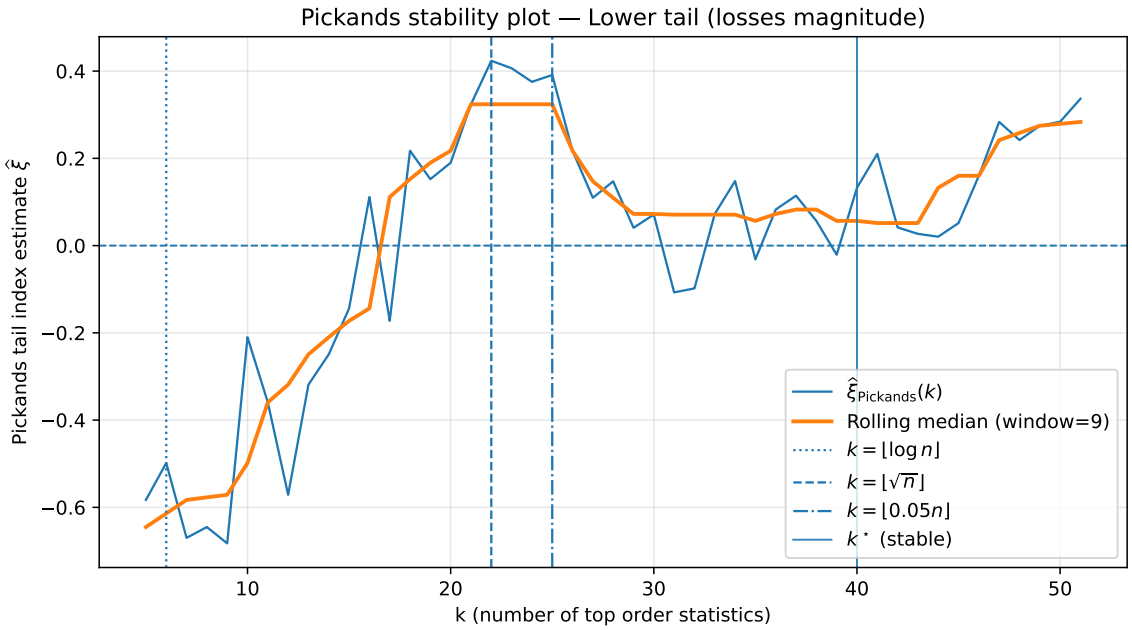

Figure 6: Pickands stability plot for the **lower tail** (loss magnitudes). The rolling median is shown as a visual aid; vertical markers indicate $k = \lfloor \log n \rfloor$, $k = \lfloor \sqrt{n} \rfloor$, $k = \lfloor 0.05n \rfloor$ and the heuristic $k^\star$.

Numerical estimates Table 3 reports the Pickands estimates for the standard threshold rules and for the proposed $k^\star$.

Tail	n	$\hat{\xi}(\lfloor \log n \rfloor)$	$\hat{\xi}(\lfloor \sqrt{n} \rfloor)$	$\hat{\xi}(k^\star)$
Upper tail (gains)	498	0.564 (at $k = 6$)	0.708 (at $k = 22$)	-0.131 (at $k^\star = 35$)
Lower tail (loss magnitudes)	519	-0.498 (at $k = 6$)	0.424 (at $k = 22$)	0.133 (at $k^\star = 40$)

Table 3: Pickands tail-index estimates for gains (upper tail) and loss magnitudes (lower tail). The dependence on k motivates the stability-plot approach.

Choice of the working threshold $k^\star$ Because the Pickands estimator is sensitive to the threshold parameter k, a single *working threshold* must be selected for EVT extrapolation. Based on the stability plots and the rolling-median smoothing, we retain the threshold located in the **most stable region of the loss-tail curve**, which provides the best bias-variance trade-off. In practice, we fix

$$\boxed{k^\star_{\text{loss}} = 40, \qquad \widehat{\xi}_{\text{loss}}(k^\star_{\text{loss}}) \approx 0.133.}$$

This value is used consistently in the remainder of the EVT-based VaR analysis (Question C.b). For the upper tail, the stability region is less clearly identified; we therefore report the estimates in Table 3 for completeness but do not rely on a single working threshold for gains.

Interpretation In the extreme-value framework, $\xi > 0$ corresponds to a **Fréchet** domain (heavy tails), $\xi = 0$ to **Gumbel**, and $\xi < 0$ to **Weibull** (bounded tail). The most robust feature in our results concerns the **downside tail**: the loss-magnitude estimates are predominantly positive over intermediate thresholds and the selected stable region yields $\hat{\xi}(k^\star_{\text{loss}}) \approx 0.13$, suggesting a **moderately heavy** lower tail. This finding is economically plausible and directly relevant for downside risk measurement. For the **upside tail**, the stability plot is less conclusive and the sign of the estimate varies with k; we therefore treat the upper-tail classification as **less robust** and focus the interpretation primarily on extreme losses.

2.3.2 b) EVT-based VaR using the Pickands estimator (losses)

We estimate an EVT-based one-day VaR on the **loss** variable $L_t = -r_t$, so that extreme losses correspond to the *upper tail* of L. Let $X_{1:n} \leq \cdots \leq X_{n:n}$ denote the order statistics of the loss sample. For a given tail size k (with $4k \leq n$), we set the threshold

$$u \;=\; X_{n-k+1:n},$$

and we use the Pickands tail-index estimate $\widehat{\xi}^P(k)$ obtained in Question C-a (stability analysis).

EVT VaR (course formula) Following the course expression for tail quantile extrapolation, the EVT VaR at confidence level α is

$$\mathrm{VaR}_\alpha^{\mathrm{EVT}}(k) = X_{n-k+1:n} + \frac{\Big(\dfrac{k}{n(1-\alpha)}\Big)^{\widehat{\xi}^P(k)} - 1}{1 - 2^{-\widehat{\xi}^P(k)}} \Big(X_{n-k+1:n} - X_{n-2k+1:n}\Big), \tag{2}$$

which is used only when extrapolation is valid, i.e. $\alpha > 1 - \frac{k}{n}$.

Implementation Listing 5 implements: (i) construction of daily losses from `rets_all`, (ii) the EVT VaR using (2) for two tail sizes ($k = \lfloor \sqrt{n} \rfloor$ and $k = k^\star$), (iii) the empirical VaR for comparison, and (iv) the figure reported in the analysis.

Listing 5: EVT one-day VaR on Natixis daily losses using the course Pickands formula (comparison with empirical VaR).

```python
# ============================================================
# Q(C-b)  EVT VaR (Pickands, course formula) on DAILY LOSSES
# Depends only on ‘rets_all‘ already built in Question A.
# ============================================================

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

assert "rets_all" in globals(), "Run Question A first to create ‘rets_all‘ (daily returns)."

# -------------
# 1) Loss sample (upper tail)
# -------------
losses = (-rets_all.loc["2015-01-02":"2018-12-31"].dropna()).astype(float).values
losses = losses[np.isfinite(losses)]
n = len(losses)

x = np.sort(losses) # order statistics X_{1:n} <= ... <= X_{n:n}

# -------------
# 2) Pickands tail index (from Q(C-a) logic)
# -------------
def pickands_xi(sample_sorted: np.ndarray, k: int) -> float:
    # sample_sorted must be sorted increasing
    n = sample_sorted.size
    if k < 1 or 4*k > n:
        return np.nan
    x1 = sample_sorted[n - k] # X_{n-k+1:n}
    x2 = sample_sorted[n - 2*k] # X_{n-2k+1:n}
    x4 = sample_sorted[n - 4*k] # X_{n-4k+1:n}
    num = x1 - x2
    den = x2 - x4
    if num <= 0 or den <= 0:
        return np.nan
    return float(np.log(num/den) / np.log(2.0))

# Two k choices (example: course-style heuristics + a selected k*)
k_sqrt = int(np.sqrt(n))
```

```python
while 4*k_sqrt >= n: k_sqrt -= 1

k_star = int(0.08*n) # replace by your selected k* from Q(C-a) if different
while 4*k_star >= n: k_star -= 1

# -------------
# 3) Course Pickands VaR formula
# -------------
def var_evt_pickands_course(sample_sorted: np.ndarray, alpha: float, k: int) -> float:
    n = sample_sorted.size
    xi = pickands_xi(sample_sorted, k)
    if not np.isfinite(xi):
        return np.nan

    # extrapolation region condition
    if alpha <= 1.0 - k/n:
        return np.nan

    u = sample_sorted[n - k] # X_{n-k+1:n}
    u2 = sample_sorted[n - 2*k] # X_{n-2k+1:n}

    denom = 1.0 - 2.0**(-xi)
    if abs(denom) < 1e-12:
        return np.nan

    factor = ( (k / (n*(1.0-alpha)))**xi - 1.0 ) / denom
    return float(u + factor * (u - u2))

# -------------
# 4) Compute VaR grid + compare to empirical
# -------------
alphas = [0.95, 0.975, 0.99, 0.995, 0.999]

rows = []
for a in alphas:
    var_emp = float(np.quantile(losses, a)) # empirical VaR on losses
    for (name, k) in [("EVT-Pickands (k_sqrt)", k_sqrt), ("EVT-Pickands (k_star)",
        k_star)]:
        var_evt = var_evt_pickands_course(x, a, k)
        rows.append({
            "alpha": a,
            "method": name,
            "k": k,
            "p_u_hat = k/n": k/n,
            "threshold u = X_{n-k+1:n}": x[n-k],
            "xi_hat (Pickands)": pickands_xi(x, k),
            "VaR_empirical": var_emp,
            "VaR_EVT (course)": var_evt,
```

```
            "extrap ok (alpha > 1-k/n)": (a > 1.0 - k/n),
        })

df_res = pd.DataFrame(rows)
display(df_res)

pivot = df_res.pivot_table(index="alpha", columns="method", values="VaR_EVT (course)")
pivot["VaR_empirical"] = [float(np.quantile(losses, a)) for a in alphas]
display(pivot)

# -------------
# 5) Figure  EVT vs empirical VaR across alpha
# -------------
plt.figure(figsize=(8.5, 4.8))
plt.plot(pivot.index, pivot["VaR_empirical"], marker="o", label="Empirical VaR (loss quantile)")
for c in pivot.columns:
    if c != "VaR_empirical":
        plt.plot(pivot.index, pivot[c], marker="o", label=c)
plt.xlabel("Confidence level ")
plt.ylabel("One-day VaR (loss units)")
plt.title("EVT-based VaR (Pickands, course formula) vs empirical VaR  Natixis daily losses")
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```

Numerical results Table 4 reports the empirical VaR (loss quantile) and the EVT extrapolations for the two tail sizes used in the code ($k = \lfloor\sqrt{n}\rfloor = 31$ and $k^\star = 86$ in our run). For $\alpha = 0.95$ and $k = \lfloor\sqrt{n}\rfloor$, extrapolation is *not* admissible because $\alpha \leq 1 - k/n$, hence no EVT value is reported in that case.

Table 4: One-day VaR on Natixis daily losses: empirical quantile vs EVT-Pickands VaR (course formula)

α	$\mathrm{VaR}_\alpha^{\mathrm{emp}}$	$\mathrm{VaR}_\alpha^{\mathrm{EVT}}$ $(k = \lfloor\sqrt{n}\rfloor = 31)$	$\mathrm{VaR}_\alpha^{\mathrm{EVT}}$ $(k = k^\star = 86)$
0.950	0.032257	-	0.032426
0.975	0.044062	0.042475	0.042858
0.990	0.055294	0.053893	0.057864
0.995	0.060226	0.061815	0.070210
0.999	0.071456	0.078090	0.102579

Graphical comparison Figure 7 summarizes the same information across confidence levels. As expected, differences between empirical and EVT-based VaR are modest for $\alpha \in [0.975, 0.99]$

and become larger in the far tail (notably at $\alpha = 0.999$), where the extrapolation is most sensitive to $\widehat{\xi}^P(k)$ and to the choice of k.

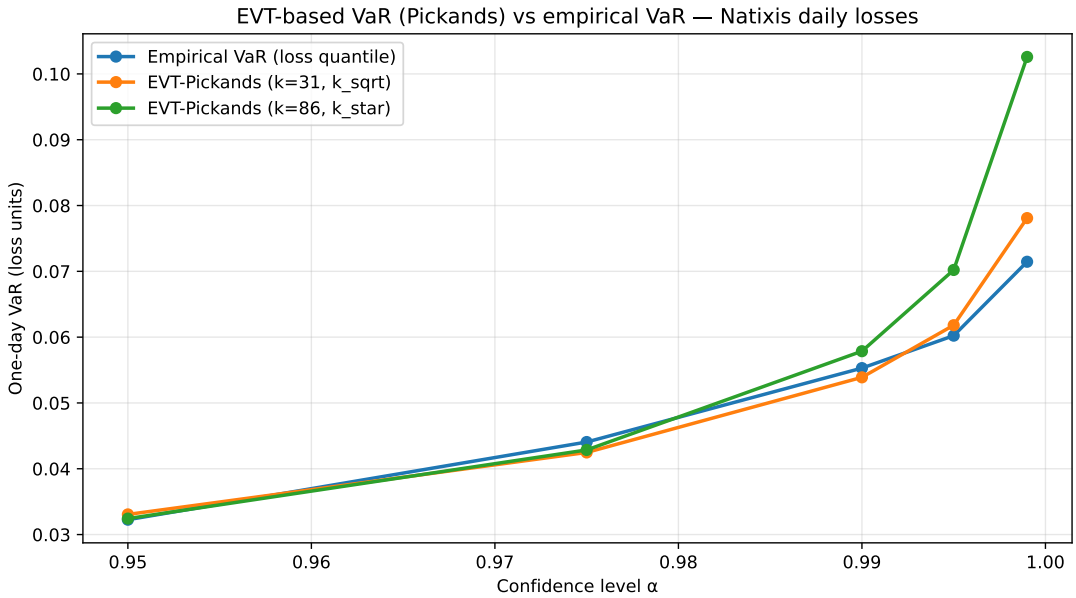

Figure 7: EVT-based VaR (Pickands, course formula) vs empirical VaR on Natixis daily losses, for two tail sizes ($k = \lfloor \sqrt{n} \rfloor$ and $k^\star$).

Conclusion This question asks whether EVT provides a meaningful extrapolation of one-day VaR beyond the empirical quantile. Our results confirm that, once the extrapolation condition $\alpha > 1 - k/n$ is satisfied, Pickands-EVT VaR tracks the empirical VaR at moderately extreme levels and produces a more conservative estimate in the far tail. However, the magnitude of the extrapolation depends materially on k (Table 4 and Figure 7), which is why the stability diagnostics of Question C-a are essential for interpreting EVT-based VaR in practice.

2.4 Question D — Estimation of Bouchaud's price impact model

2.4.1 Theoretical background

Bouchaud's model, presented in the *Market Risk 2025-2026* lecture notes (section *Liquidity Risk - Price Impact Models*), describes price formation as the sum of the impacts of past orders:

$$p_t = p_{-\infty} + \sum_{s<t} G(t-s)\, \varepsilon_s\, S_s\, V_s^r, \tag{3}$$

where:

- $\varepsilon_s \in \{-1, +1\}$ is the transaction sign ($+1$ for buyer-initiated trades, -1 for seller-initiated trades);
- S_s is the bid-ask spread at time s;
- V_s is the transaction volume;

- $r \in [0,1]$ controls the concavity of the volume-impact function;
- $G(\tau)$ is the *impact kernel* or *propagator*, decreasing in τ.

Differentiating (3) yields the operational form of the model for price changes:

$$\Delta p_t = p_{t+1} - p_t = \sum_{\ell=0}^{L} G(\ell)\, \varepsilon_{t-\ell}\, S_{t-\ell}\, V_{t-\ell}^{r} + \xi_t, \tag{4}$$

where ξ_t denotes an unexplained residual noise term.

This model captures a **transient impact**: trades affect prices immediately, and the effect then decays over time.

2.4.2 Estimation methodology

Using the provided dataset (TD4), we estimate the coefficients $G(\ell)$ and the exponent r in (4) with the following procedure:

1. **Variable construction:**
   - p_t = price before the transaction;
   - ε_t = transaction sign;
   - S_t = bid-ask spread;
   - $\Delta p_t = p_{t+1} - p_t$.
2. **Volume treatment (normalization)** Since volume is very often missing in the dataset, we normalize observed volumes by their median and impute missing volumes with 1, which amounts to imputing a *typical* volume:

   $$V_t^{\text{scaled}} = \frac{V_t}{\text{median}(V \mid V \text{ observed})}, \qquad V_t^{\text{scaled}} = 1 \text{ if } V_t \text{ is missing.}$$

   In the remainder, V_t denotes V_t^{scaled}.
3. **Construction of the instantaneous impact regressor:**

   $$x_t = \varepsilon_t\, S_t\, V_t^{\,r}. \tag{5}$$

   We then build the lagged versions $x_{t-\ell}$ for $\ell = 0, \ldots, L$.
4. **Linear regression (for a fixed r):**

   $$\widehat{\mathbf{G}}(r) = \arg\min_{\mathbf{G}} \|\Delta\mathbf{p} - X\mathbf{G}\|^2,$$

   where X contains the columns $(x_t, x_{t-1}, \ldots, x_{t-L})$.
5. **Optimization over r:** we vary r over a grid in $[0,1]$ (step 0.05) and retain the value that maximizes the regression R^2.

The following code implements these steps.

Listing 6: Estimation of Bouchaud's impact model (TD4) with normalized volume

```python
import numpy as np
import pandas as pd

df_b = df.copy()

# Price p_t (price before transaction)
df_b["p"] = df_b["Price (before transaction)"].astype(float)

# Sign epsilon_t
df_b["eps"] = df_b["Sign of the transaction"].astype(float)

# Bid-ask spread S_t
df_b["S"] = df_b["bid-ask spread"].astype(float)

# Volume: V_scaled = V / median(V|observed), and V_scaled=1 if volume is missing
V_raw = df_b["volume of the transaction (if known)"].astype(float)
V_med = V_raw.dropna().median()
df_b["V_scaled"] = (V_raw / V_med).fillna(1.0).astype(float)

# Price changes: p_t = p_{t+1} - p_t
df_b["dp"] = df_b["p"].shift(-1) - df_b["p"]
df_b = df_b.iloc[:-1].copy()

# Parameters
L = 10
r_grid = np.linspace(0.0, 1.0, 21)

best_R2 = -np.inf
best_r = None
best_G = None

for r_exp in r_grid:
    df_tmp = df_b.copy()

    # Impact regressor x_t = eps_t * S_t * (V_scaled)^r
    df_tmp["x"] = df_tmp["eps"] * df_tmp["S"] * (df_tmp["V_scaled"] ** r_exp)

    # Lags x_{t-l}
    for ell in range(L + 1):
        df_tmp[f"x_lag_{ell}"] = df_tmp["x"].shift(ell)

    df_tmp = df_tmp.iloc[L:].copy()

    y = df_tmp["dp"].values
    X_cols = [f"x_lag_{ell}" for ell in range(L + 1)]
```

```
    X = df_tmp[X_cols].values

    # OLS regression
    G_hat, _, _, _ = np.linalg.lstsq(X, y, rcond=None)

    y_hat = X @ G_hat
    resid = y - y_hat

    ss_tot = np.sum((y - y.mean())**2)
    ss_res = np.sum(resid**2)
    R2 = 1 - ss_res / ss_tot

    if R2 > best_R2:
        best_R2 = R2
        best_r = r_exp
        best_G = G_hat

df_G = pd.DataFrame({"lag": np.arange(L + 1), "G_hat": best_G})

print(df_G)
print(f"Volume median (observed): {V_med:.6f}")
print(f"Best r : {best_r:.3f}")
print(f"Best R2 : {best_R2:.6f}")
```

The following code generates the two plots (full kernel and zoom on the lagged part $\ell \geq 1$) reported in the Results section (Figures 8 and 9).

Listing 7: Generating plots of the estimated impact kernel

```
import matplotlib.pyplot as plt

lags = df_G["lag"].values
G_hat = df_G["G_hat"].values

plt.figure()
plt.stem(lags, G_hat)
plt.xlabel("Lag l")
plt.ylabel("G_hat(l)")
plt.title("Q5  Estimated Bouchaud impact kernel")
plt.tight_layout()
plt.show()

plt.figure()
plt.stem(lags[1:], G_hat[1:])
plt.axhline(0, linewidth=1)
plt.xlabel("Lag l (l >= 1)")
plt.ylabel("G_hat(l)")
plt.title("Q5  Lagged impact (l >= 1)")
plt.tight_layout()
```

```
plt.show()
```

2.4.3 Results

With the volume normalization described above, we obtain:

$$\text{median}(V \mid V \text{ observed}) = 77, \qquad \hat{r} = 0.55, \qquad R^2 = 0.768859.$$

Table 5 reports the estimated kernel:

Lag ℓ	$\widehat{G}(\ell)$
0	0.615540
1	−0.033182
2	−0.020196
3	−0.002006
4	0.008213
5	0.000728
6	−0.003912
7	−0.005140
8	−0.017402
9	−0.011012
10	−0.024505

Table 5: Estimated impact kernel $\widehat{G}(\ell)$ (normalized volume, $L = 10$)

Figures 8 and 9 illustrate the fast decay of the estimated impact kernel (figures produced using the *normalized-volume* specification).

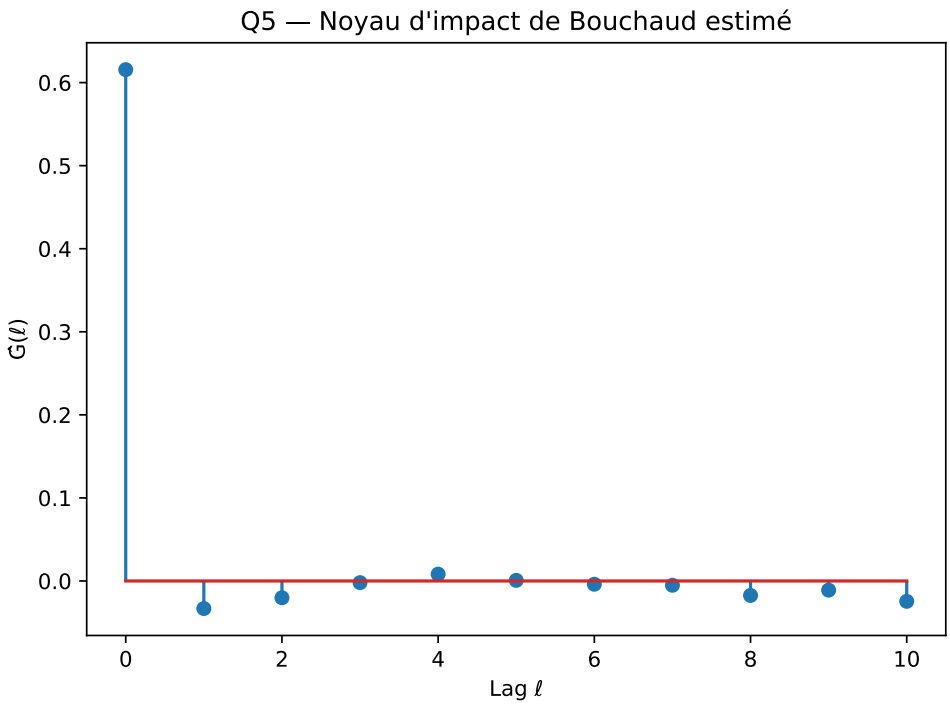

Figure 8: Estimated impact kernel $\widehat{G}(\ell)$ for $\ell = 0, \ldots, L$ (normalized volume).

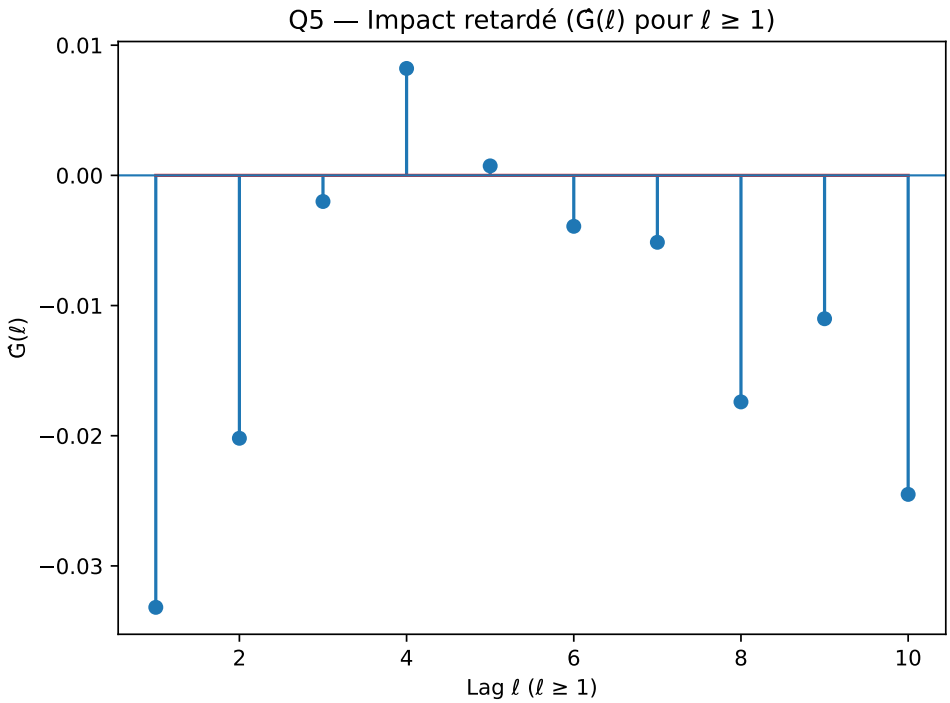

Figure 9: Zoom on the lagged impact $\widehat{G}(\ell)$ for $\ell \geq 1$ (normalized volume).

2.4.4 Analysis and interpretation

Instantaneous impact. The coefficient $\widehat{G}(0) \approx 0.616$ is dominant: the price change is mostly explained by the immediate impact of the current transaction. The strong concentration of

the impact at $\ell = 0$ is consistent with a microstructure setting in which impact is primarily instantaneous at the transaction time scale.

Lagged impact. The lagged coefficients $\widehat{G}(\ell)$ for $\ell \geq 1$ have small magnitude (order 10^{-2}) and are mostly negative. This suggests a *partial reversion* after the initial impact: the effect of the transaction dissipates quickly and opposite-signed corrections appear at some lags, which remains compatible with the idea of a transient impact.

Volume dependence We obtain $\hat{r} = 0.55$, which corresponds to a concave dependence close to a *square-root* type law (values around 0.5 are frequently discussed empirically). However, the interpretation of r should remain cautious: the high fraction of missing volumes implies that a large share of observations uses $V_t^{\text{scaled}} = 1$ (typical volume), which limits fine identification of r despite normalization.

Goodness of fit The explanatory power of the model is high ($R^2 \simeq 0.769$). Part of this fit may come from the definition of p_t (the field "Price (before transaction)" may be close to a transaction price). In that case, spread-related microstructure effects can amplify the apparent impact. We therefore interpret this R^2 with caution.

2.4.5 Conclusion

The empirical estimation of Bouchaud's price impact model highlights:

- a **predominant instantaneous impact**, with $\widehat{G}(0) \approx 0.616$;
- a **fast decay** of the kernel, with lagged terms of small magnitude (order 10^{-2});
- a **concave (sub-linear) volume dependence**, with $\hat{r} \approx 0.55$ after normalizing volume by its median.

These results are qualitatively consistent with the propagator intuition: the impact of a trade is mainly instantaneous and dissipates quickly. The main empirical limitation of our estimation is the large proportion of missing volumes, which reduces the fine identifiability of the exponent r. We therefore interpret $\hat{r}$ with caution, while the overall shape of the kernel $\widehat{G}(\ell)$ appears robust and economically plausible. In light of these elements, the model is **qualitatively well specified** to describe a transient impact on this dataset, but the calibration of the "volume" component remains **less reliable** due to incomplete information on V_t.

2.5 Question E(a) — Multiresolution correlations (Haar wavelets) and the Epps effect

Using the TD5 dataset, we quantify the dependence structure between three FX rates (GBPEUR, SEKEUR and CADEUR) across time scales with Haar wavelets. The statement requires: (i) constructing an average price from HIGH/LOW, (ii) converting it into returns at the finest sampling step, and (iii) computing multiresolution correlations for all FX pairs. We then evaluate whether an *Epps effect* is observed and provide microstructure-based explanations.

2.5.1 Average price and high-frequency returns

We work with the required average price proxy:

$$P_t^{\text{avg}} \;=\; \frac{\text{HIGH}_t + \text{LOW}_t}{2}, \tag{6}$$

and construct log-returns at the smallest available time step (about 15 minutes):

$$r_t \;=\; \log(P_t^{\text{avg}}) - \log(P_{t-1}^{\text{avg}}). \tag{7}$$

This yields $N = 12{,}928$ high-frequency returns. Since a full Haar DWT is most convenient on dyadic sample sizes, we truncate to the largest power of two, $N_{\text{pow2}} = 8{,}192$, which allows a decomposition depth of $J_{\max} = 13$ levels.

2.5.2 Haar multiresolution decomposition

For each return series, we apply the multi-level Haar DWT (TD5-Q1) and extract the detail coefficients

$$\{c_{j,k}^{(X)}\}_{k=1}^{n_j}, \qquad j = 1, \ldots, J_{\max},$$

where $j = 1$ corresponds to the finest scale (about 15 minutes) and larger j correspond to coarser horizons. With base step $\Delta t \approx 15$ minutes, the time scale associated with level j is approximately

$$\Delta t_j \;\approx\; 2^{j-1}\Delta t. \tag{8}$$

2.5.3 Multiresolution covariance and correlation (course definition)

Following the multiscale framework, the *multiresolution covariance* between two assets X and Y at level j is estimated from their wavelet coefficients at that scale:

$$\widehat{\text{Cov}}_{XY}(j) \;=\; \frac{1}{n_j}\sum_{k=1}^{n_j}\Big(c_{j,k}^{(X)} - \overline{c}_j^{(X)}\Big)\Big(c_{j,k}^{(Y)} - \overline{c}_j^{(Y)}\Big), \tag{9}$$

with $\overline{c}_j^{(X)} = \frac{1}{n_j}\sum_{k=1}^{n_j} c_{j,k}^{(X)}$. The corresponding *multiresolution correlation* is:

$$\widehat{\rho}_{XY}(j) \;=\; \frac{\widehat{\text{Cov}}_{XY}(j)}{\sqrt{\widehat{\text{Var}}_X(j)}\sqrt{\widehat{\text{Var}}_Y(j)}}, \qquad \widehat{\text{Var}}_X(j) = \widehat{\text{Cov}}_{XX}(j). \tag{10}$$

In our implementation, we compute $\widehat{\rho}_{XY}(j)$ directly as the sample correlation of the Haar *detail coefficients* $(c_{j,k}^{(X)})_k$ and $(c_{j,k}^{(Y)})_k$ for each pair $(X, Y) \in \{\text{GBP-SEK}, \text{GBP-CAD}, \text{SEK-CAD}\}$ and each level j.

2.5.4 Results: multiresolution correlations and Epps effect

Figure 10 reports the estimated multiresolution correlations across wavelet levels. In the discussion, we focus primarily on levels $j = 1$ to $j = 9$ (and, at most, $j = 10$ with caution), since higher levels contain too few coefficients for reliable correlation estimation.

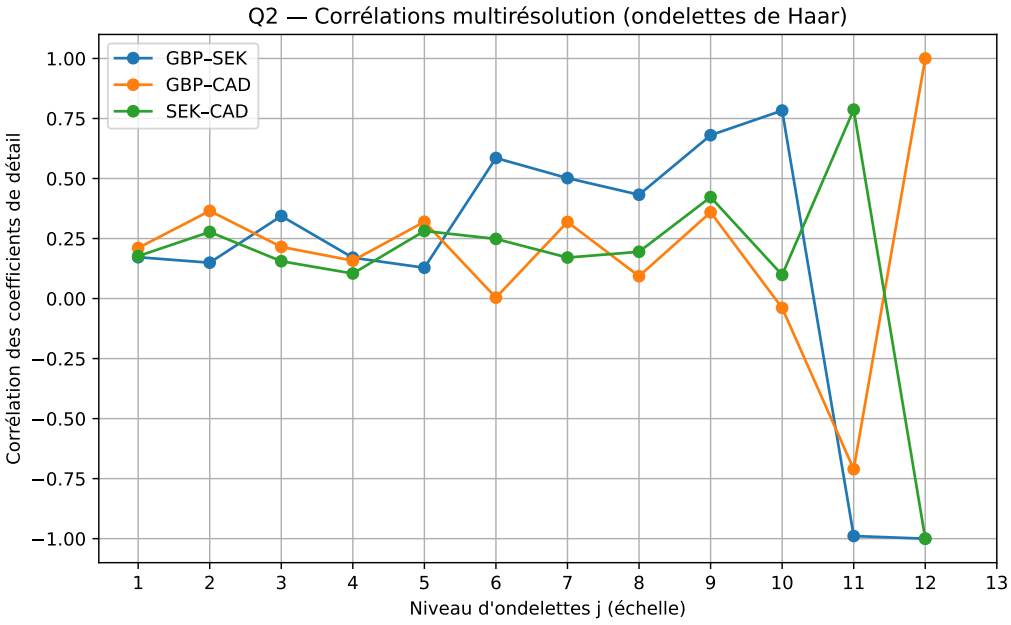

Figure 10: Multiresolution correlations computed from Haar detail coefficients across wavelet levels (base step $\Delta t \approx 15$ minutes).

High-frequency horizons (levels 1-4) At the finest resolutions (15 to 120 minutes), correlations are *positive but moderate*. In our estimates, GBP-SEK typically lies around 0.15-0.34, SEK-CAD around 0.10-0.28, while GBP-CAD remains mostly below about 0.40. This indicates that at very short horizons, the three FX series are not perfectly synchronized and that idiosyncratic high-frequency fluctuations remain important.

Intermediate horizons (levels 5-9): evidence consistent with an Epps effect. When moving to coarser time scales (roughly from a few hours up to a few days), correlations become more pronounced for some pairs, especially GBP-SEK and SEK-CAD. For example, GBP-SEK reaches values around 0.58 (level 6) and about 0.68 (level 9), while SEK-CAD increases up to about 0.42 (level 9). Overall, the pattern of *lower correlations at very high frequency and higher correlations at aggregated horizons* is consistent with the **Epps effect** commonly observed in high-frequency data.

Very coarse horizons (levels ≥ 10): unreliable estimates At the largest scales, the estimates become unstable and can take extreme values close to ± 1 (or be undefined). This behavior is not economically robust: the number of detail coefficients halves at each level, so the highest levels contain very few observations (e.g. 8 at level 10, 4 at level 11, 2 at level 12 and 1 at level 13). Correlations at these levels are therefore dominated by finite-sample noise and should not be interpreted.

2.5.5 How to explain the Epps effect (microstructure mechanisms)

The Epps effect can be explained by standard market microstructure arguments, consistent with our multiscale results:

- **Asynchronous sampling (non-synchronicity):** FX pairs are not recorded at exactly the same instants. At very high frequency, this induces a "timing mismatch" that mechanically lowers measured correlations.
- **Microstructure noise:** bid-ask bounce, liquidity heterogeneity, and measurement noise affect short-horizon returns more strongly, but tend to average out as the horizon increases.
- **Stronger common components at longer horizons:** at horizons of hours/days, FX rates are increasingly driven by shared macro/market factors (EUR moves, global risk, macro announcements), which increases co-movement and therefore correlations.

Answer to the question Yes, we observe an Epps-type behavior: correlations are modest at the finest horizons (15-120 minutes) and become larger at intermediate horizons (hours to days), while the coarsest levels are not interpretable due to the very small number of wavelet coefficients.

2.6 Question E(b) — Hurst exponents and annualized volatilities

This question aims at (i) estimating the Hurst exponent H for each FX rate (GBPEUR, SEKEUR, CADEUR) and (ii) computing annualized volatilities using the *daily* volatility together with the Hurst-scaling rule, as introduced in the course.

Methodological framework Departures from the Brownian scaling $\sqrt{\delta}$ are captured through a fractional scaling δ^H, where $H \in [0,1]$ is the Hurst exponent. In particular, for increments over a time scale δ,

$$\mathrm{Var}(\Delta X_\delta) \propto \delta^{2H} \qquad \Longrightarrow \qquad \sigma(\delta) \propto \delta^H.$$

We estimate H via the classical rescaled-range (R/S) method. For a window size w, the statistic satisfies

$$\frac{R(w)}{S(w)} \propto w^H,$$

so that H is obtained as the slope of a linear regression of $\log(R/S)$ on $\log(w)$.

For volatility annualization, we follow the course rule: the classical Brownian annualization is

$$\sigma_{\text{ann}}^{(0.5)} = \sigma_{\text{daily}}\sqrt{252},$$

while the Hurst-adjusted annualization is

$$\sigma_{\text{ann}}^{(H)} = \sigma_{\text{daily}}\, 252^H \qquad \Longleftrightarrow \qquad \sigma_{\text{ann}}^{(H)} = \sigma_{\text{ann}}^{(0.5)}\, 252^{H-\frac{1}{2}}.$$

Implementation (Python) The following cell code implements the R/S estimator (computed on intraday returns for statistical stability) and the annualization based on the *daily* volatility.

Listing 8: Hurst exponents (R/S) and annualized volatilities with Hurst scaling

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```python
from scipy import stats

pairs = ["GBPEUR", "SEKEUR", "CADEUR"]

# Intraday (15-min) returns for H estimation (mid_prices built as in Q2)
returns_15m = np.log(mid_prices).diff().dropna()

def hurst_exponent_rs(series):
    series = np.asarray(series, dtype=float)
    N = len(series)

    window_sizes = np.floor(np.logspace(2, np.log10(N), 10)).astype(int)
    window_sizes = np.unique(window_sizes)

    RS, valid_w = [], []
    for w in window_sizes:
        n_segments = N // w
        if n_segments < 1:
            continue
        rs_vals = []
        for i in range(n_segments):
            segment = series[i*w:(i+1)*w]
            Z = segment - segment.mean()
            Y = np.cumsum(Z)
            R = Y.max() - Y.min()
            S = np.std(segment)
            if S > 0:
                rs_vals.append(R / S)
        if len(rs_vals) > 0:
            RS.append(np.mean(rs_vals))
            valid_w.append(w)

    log_w = np.log(valid_w)
    log_RS = np.log(RS)
    slope, intercept, _, _, _ = stats.linregress(log_w, log_RS)
    return slope, np.array(valid_w), np.array(RS)

# Estimate H and plot
hurst_values = {}
plt.figure(figsize=(10,6))
for c in pairs:
    H, w, RS = hurst_exponent_rs(returns_15m[c].values)
    hurst_values[c] = H
    plt.plot(np.log(w), np.log(RS), "o-", label=f"{c} (H{H:.3f})")
plt.xlabel("log(window size)")
plt.ylabel("log(R/S)")
plt.title("Hurst exponent estimation (R/S)")
plt.legend()
```

```
plt.grid(True)
plt.tight_layout()
plt.show()

# Daily volatility from daily returns
daily_prices = mid_prices.resample("1D").last().dropna()
daily_returns = np.log(daily_prices).diff().dropna()
sigma_daily = daily_returns.std()

# Annualization
sigma_ann_brownian = sigma_daily * np.sqrt(252)
sigma_ann_hurst = pd.Series({c: sigma_daily[c] * (252 ** hurst_values[c]) for c in
    pairs})
```

Results Figure 11 reports the log-log R/S curves, from which we obtain:

$$\widehat{H}_{\text{GBPEUR}} = 0.5532, \qquad \widehat{H}_{\text{SEKEUR}} = 0.4722, \qquad \widehat{H}_{\text{CADEUR}} = 0.4432.$$

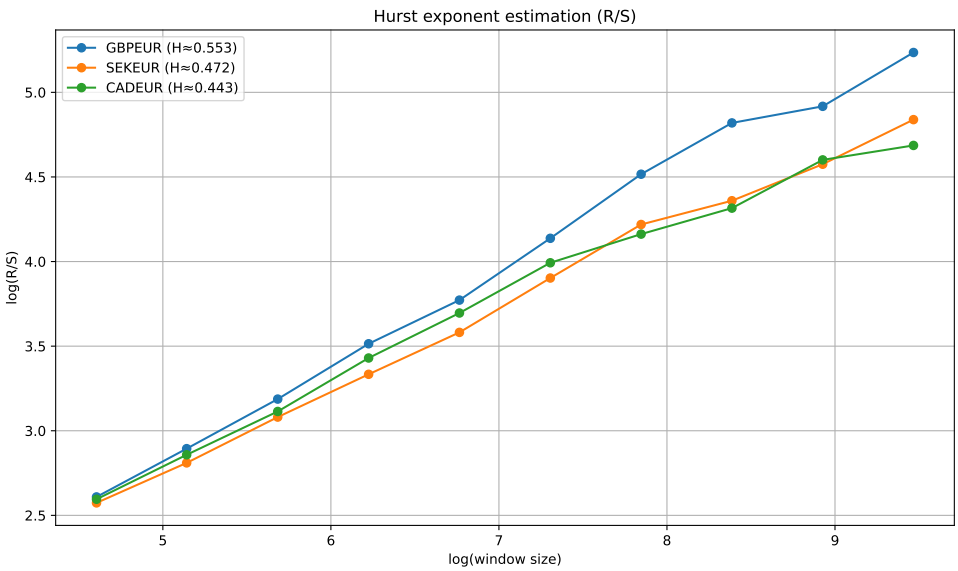

Figure 11: Hurst exponent estimation using the R/S method (log-log regression).

Daily volatilities and annualized volatilities are reported in Table 6.

FX rate	$\widehat{H}$	σ_{daily}	$\sigma_{\text{ann}}^{(0.5)}$	$\sigma_{\text{ann}}^{(H)}$
GBPEUR	0.5532	0.007828	0.124271	0.166791
SEKEUR	0.4722	0.003012	0.047816	0.041009
CADEUR	0.4432	0.005164	0.081978	0.059875

Table 6: Hurst exponents and annualized volatilities. The Hurst-based annualization uses $\sigma_{\text{ann}}^{(H)} = \sigma_{\text{daily}}\, 252^{H}$.

Interpretation GBPEUR exhibits mild persistence ($\widehat{H} > 0.5$), whereas SEKEUR and CADEUR are anti-persistent ($\widehat{H} < 0.5$). As a result, the Hurst correction increases the annualized volatility for GBPEUR and decreases it for SEKEUR and CADEUR relative to the Brownian benchmark. This highlights that a unique $\sqrt{t}$ scaling may misstate long-horizon risk when the empirical scaling exponent differs from 1/2.

3 Conclusion

This project illustrates why modern market risk analysis cannot rely on a single risk metric nor on a single statistical framework. Starting from a fully non-parametric perspective, the biweight-KDE historical VaR provides a transparent and assumption-light estimate of one-day downside risk. Its out-of-sample backtest over the 2017-2018 period exhibits a conservative behavior, with fewer violations than the nominal rate. While this conservatism is desirable from a risk-management standpoint, it also indicates that historical VaR may be imperfectly calibrated under standard unconditional coverage diagnostics.

Extending the analysis beyond quantile-based measures, the Expected Shortfall is systematically larger than the corresponding VaR, confirming that VaR alone understates the average severity of extreme losses.

Extreme Value Theory further refines tail-risk measurement by enabling extrapolation beyond the empirical sample. Using the Pickands estimator, we find clear evidence of asymmetry between gains and losses, with moderately heavy tails on the loss side and less robust tail behavior on the gain side. EVT-based VaR estimates become substantially more conservative at high confidence levels, but also display sensitivity to the threshold choice, making stability diagnostics and bias-variance trade-offs essential in practice. Throughout the EVT analysis, returns are assumed to be independent and identically distributed; while this assumption is standard in first-order EVT, it abstracts from volatility clustering and should therefore be kept in mind when interpreting far-tail extrapolations.

Beyond distributional risk, the liquidity analysis based on Bouchaud's price impact model suggests that price impact is predominantly instantaneous, with a fast-decaying transient component and a concave dependence on traded volume. These findings are consistent with market microstructure theory and course intuition, while also highlighting practical limitations because of incomplete information on trading volumes.

Finally, the multiscale wavelet analysis reveals a scale-dependent dependence structure between FX rates, consistent with the presence of an Epps effect at high frequencies. The estimated Hurst exponents indicate that volatility may scale as t^H rather than $t^{1/2}$, implying that standard annualization rules can either overstate or understate long-horizon risk depending on the persistence properties of the series.

Overall, this project demonstrates that no single method is enough for comprehensive market risk assessment. Non-parametric VaR is well suited for routine short-horizon monitoring, EVT-based methods are indispensable for far-tail risk evaluation, and liquidity and multiscale analyses provide essential complementary dimensions. Combining these approaches leads into a more robust, interpretable, and economically meaningful representation of market risk than any individual model taken in isolation.